

TALLER 4 - IA

Nicolas Valentin Avendaño-202222719

Johan Sebastian Castiblanco Galeano - 202316250

Daniela Martinez 202310127

Nicolas Godoy -

Punto 2

Al probar forwardBFS en distintos layouts simples, se observa que el algoritmo encuentra planes válidos, pero a medida que el mapa se hace más grande, o existen mas acciones posibles, el número de estados explorados aumenta. Esto dado a que BFS, explora por niveles, primero revisando todos los planes de longitud 1, luego longitud 2, y así sucesivamente, hasta que encuentra el objetivo, garantizando el plan mas corto.

Punto 3

d).

La búsqueda regresiva suele ser mas eficiente en problemas en donde el objetivo final esta definidos claramente, en estos casos el algoritmo no debe explorar todas las acciones posibles, y selecciona y expande unicamente las acciones que pueden contribuir en llegar al objetivo.

Por ejemplo, el objetivo Rescued(patient_0) dirige la accion hacia rescue, y posteriormente hacia sus precondiciones, haciendo que la búsqueda sea mas enfocada que un forward checking.

Sin embargo, si hay muchos objetivos, podría ser menos eficiente, ya que podrían generarse demasiados subobjetivos inconsistentes que podrían hacer que sea menos eficiente.

Punto 4.

a).

IgnorePreconditions:

Esta heurística consiste en ignorar TODAS las precondiciones, por lo cual todas las acciones en teoria se pueden ejecutar a la vez. En la práctica es un escenario irreal, pero es un buen punto de partida.

Hay que recordar que para que una heurística sea admisible la misma no debe sobreestimar el costo real, sino ser optimista al mismo respecto al objetivo.

Probando dicha heurística en con el algoritmo de planificación astarPlanner en TinyBase se obtiene el siguiente resultado:



```

=====
Operación Fénix - ISIS-1611 Inteligencia Artificial
=====
Layout: tinyBase (5x7)
Problema: SimpleRescueProblem
Pacientes: ['patient_0']
Suministros: ['supplies_0']
Puestos médicos: [(1, 2)]
=====
Planificador: aStarPlanner
Heurística: ignorePreconditions

Tiempo de planificación: 0.853s
Estados expandidos: 252
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...

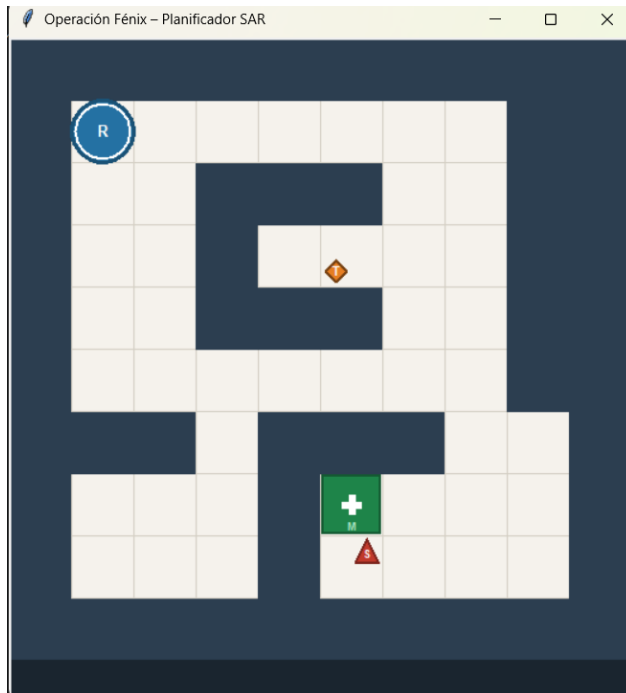
¡Misión completada exitosamente!

```

Al analizar la secuencia de acciones, el plan que hace es el más optimo, primero moviendose a los suministros médicos, recogiendo los, dejándolos en el objetivo (que queda de camino

hacia el paciente) luego va por el paciente, lo recoge y lo atiende en el punto médico. Ello demuestra que la heurística es optimista y no sobreestima el costo de llegar al objetivo.

Ahora probando la heurística en smallRescue:



```
Planificador: aStarPlanner
Heurística: ignorePreconditions

Tiempo de planificación: 24.680s
Estados expandidos: 1316
Longitud del plan: 23 acciones

Plan:
  1. Move(robot, (1, 8), (2, 8))
  2. Move(robot, (2, 8), (3, 8))
  3. Move(robot, (3, 8), (4, 8))
  4. Move(robot, (4, 8), (5, 8))
  5. Move(robot, (5, 8), (6, 8))
  6. Move(robot, (6, 8), (6, 7))
  7. Move(robot, (6, 7), (6, 6))
  8. Move(robot, (6, 6), (5, 6))
  9. Pickup(robot, supplies_0, (5, 6))
 10. Move(robot, (5, 6), (6, 6))
 11. Move(robot, (6, 6), (6, 5))
 12. Move(robot, (6, 5), (6, 4))
 13. Move(robot, (6, 4), (7, 4))
 14. Move(robot, (7, 4), (7, 3))
 15. Move(robot, (7, 3), (7, 2))
 16. Move(robot, (7, 2), (6, 2))
 17. Move(robot, (6, 2), (5, 2))
 18. SetupSupplies(robot, supplies_0, (5, 2))
 19. Move(robot, (5, 2), (5, 1))
 20. Pickup(robot, patient_0, (5, 1))
 21. Move(robot, (5, 1), (5, 2))
 22. PutDown(robot, patient_0, (5, 2))
 23. Rescue(robot, patient_0, (5, 2))
```

Ejecutando plan...

¡Misión completada exitosamente!

De igual forma, la heurística permite que el algoritmo cumpla con la misión siguiendo la ruta más optima, siendo admisible.

b).

Es interesante conocer el desempeño de las heurísticas implementadas contra un modelo sin heurística, por ejemplo, el forwardBFS. Aquí tomando como base el layout tinyBase se obtiene lo siguiente:

```
Planificador: aStarPlanner
Heurística: ignorePreconditions

Tiempo de planificación: 0.925s
Estados expandidos: 252
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...

Planificador: aStarPlanner
Heurística: ignoreDeleteLists

Tiempo de planificación: 0.182s
Estados expandidos: 25
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...
¡Misión completada exitosamente!

Planificador: forwardBFS

Tiempo de planificación: 0.023s
Estados expandidos: 225
Longitud del plan: 9 acciones

Plan:
1. Move(robot, (1, 4), (1, 3))
2. Pickup(robot, supplies_0, (1, 3))
3. Move(robot, (1, 3), (1, 2))
4. SetupSupplies(robot, supplies_0, (1, 2))
5. Move(robot, (1, 2), (1, 1))
6. Pickup(robot, patient_0, (1, 1))
7. Move(robot, (1, 1), (1, 2))
8. PutDown(robot, patient_0, (1, 2))
9. Rescue(robot, patient_0, (1, 2))

Ejecutando plan...
¡Misión completada exitosamente!
```

Basados en estos resultados, se puede inferir lo siguiente:

1. **A* con ignoreDeleteLists (25 estados):** Es el algoritmo más eficiente en cuanto a la exploración del espacio de búsqueda. Logró encontrar el plan óptimo expandiendo un número drásticamente menor de estados en comparación con la búsqueda no informada (BFS).

2. **forwardBFS sin heurística (225 estados):** Al ser un algoritmo de búsqueda ciega (Primero a lo Ancho), explora el espacio de estados de manera sistemática y exhaustiva. Como era de esperarse, expande muchos más estados que un A* bien guiado.
3. **A* con ignorePreconditions (252 estados):** Esta heurística resultó tener el peor desempeño en términos de estados expandidos, superando incluso a la búsqueda ciega (225). Esto indica que la estimación que hace esta heurística es de muy baja calidad, lo que provoca que el algoritmo evalúe demasiados nodos innecesarios antes de llegar a la meta. Que un A* expanda más nodos que BFS suele ocurrir cuando la heurística devuelve valores muy bajos (ceranos a 0, comportándose como un algoritmo de costo uniforme) y la forma en la que resuelve los empates en la frontera es menos eficiente que el orden natural de un BFS.

En conclusión, la heurística ignoreDeleteLists reduce significativamente el espacio de búsqueda, mientras que ignorePreconditions es ineficiente y rinde peor que no usar ninguna heurística.

Información entregada por las heurísticas:

La heurística más informativa es IgnoreDeleteLists.

Se dice que una heurística h_1 es más informativa que otra h_2 si para todo nodo n , $h_1(n)$ es mayor o igual que $h_2(n)$. En la práctica, esto se traduce directamente en la cantidad de estados que el algoritmo necesita expandir. Los resultados muestran que ignoreDeleteLists expandió solo 25 estados, mientras que ignorePreconditions expandió 252 estados. Al expandir un 90% menos de estados, queda demostrado empíricamente que ignoreDeleteLists guía la búsqueda de forma mucho más certera hacia la meta.

Teóricamente, en un dominio de rescate, las precondiciones son fundamentales:

- Al usar ignorePreconditions, el planificador asume que puede hacer cualquier acción en cualquier momento. Esto hace que el problema relajado sea demasiado fácil, devolviendo valores heurísticos bajísimos que no reflejan el costo real.
- Por otro lado, ignoreDeleteLists asume que una vez que se logra algo (ej. tener un suministro, moverse a una posición), nunca lo pierde. Esta relajación es mucho más realista, ya que mantiene la estructura lógica de los pasos requeridos (la cadena de precondiciones), dando como resultado una estimación del costo restante mucho más cercana a la realidad y, por ende, una heurística más alta e informativa.

Punto 5

El orden en el que se procesan los pacientes afecta directamente la cantidad de pasos totales. Aunque en nuestra jerarquía HTN (basada en *FullRescueMission*) el robot suele retornar al Puesto Médico al finalizar cada rescate individual, el desplazamiento inicial es variable. Dependiendo de a qué paciente o suministro se le asigne prioridad, la distancia recorrida desde la posición de inicio hasta el primer objetivo cambiará, resultando en planes de diferente longitud.

Se presenta un alto costo computacional el cual radica en la forma en la que el algoritmo explora el espacio de soluciones. Al utilizar una Búsqueda a lo Ancho (BFS) para refinar la jerarquía HTN, el algoritmo explora todas las posibles secuencias de refinamiento nivel por nivel. En escenarios con múltiples pacientes, la cantidad total de acciones primitivas necesarias (la profundidad del plan) es muy alta. Como BFS tiene un crecimiento exponencial espacial y temporal, la cola de planes parciales generada rápidamente satura la memoria y el tiempo de ejecución antes de encontrar el estado meta.

Punto 6

Forward Planning

Complejidad Temporal

La complejidad temporal es

$$O(b^d)$$

Donde b corresponde al número de acciones relevantes, y d a la profundidad antes de encontrar el objetivo, esto dado a que BFS puede explorar todos los estados disponibles hasta profundidad d antes de encontrar el objetivo.

Complejidad Espacial

Se almacenan el conjunto de estados visitados y la frontera, también se guarda el plan parcial asociado.

$$O(b^d)$$

Complejidad

El algoritmo es completo si el espacio de estados es finito, y se evita repetir estados, si existe un plan válido, el algoritmo eventualmente lo encontrará.

Optimalidad

Es óptimo cuando todas las acciones tienen el mismo costo, como explora primero los menores costos, siempre encontrará primero la solución con menor número de acciones, con costos diferentes ya no se garantiza las menores acciones, ya que solo minimiza la profundidad.

Backward Planning

Complejidad Temporal

La complejidad temporal es

$$O(b^d)$$

Donde b corresponde al número de acciones relevantes, y d a la profundidad del algoritmo, en el peor de los casos se explora una cantidad exponencial de objetivos.

Complejidad Espacial

Se almacenan los objetivos visitados, y la frontera de búsqueda por lo cual también será

$$O(b^d)$$

Complejidad

El algoritmo es completo ya que explora sistemáticamente el espacio de objetivos, y evita ciclos al utilizar una estructura de visitados, si el espacio de búsqueda es finito, eventualmente se encontrará una solución

Optimalidad

Se utiliza una estrategia tipo BFS sobre el espacio de objetivos, por lo que bajo costos uniformes, será óptimo.

A* Planning:

La complejidad temporal es, en el peor de los casos $O(b^d)$, donde b es el factor de ramificación (número promedio de ramas por nodo) y d es la profundidad de la solución óptima. En el mejor de los casos es $O(b)$, donde la heurística es tan buena que permite que siga solo los nodos óptimos.

Complejidad Espacial

Aquí sin importar la optimalidad de la heurística, siempre será $O(b^d)$, ya que A* siempre almacena los nodos recorridos en memoria.

Complejidad

El algoritmo es completo ya que explora sistemáticamente el espacio de objetivos, y evita ciclos al utilizar una estructura de visitados, si el espacio de búsqueda es finito, eventualmente se encontrará una solución

Optimalidad

Es importante recalcar que A* se basa en Dijkstra, el cual siempre encuentra el camino más óptimo gracias a su guardado en memoria de nodos con comparación de nodos y heurística.

Reflexión sobre el uso de la IA

Para la realización de este taller se utilizó ayuda de la Inteligencia artificial, El principal uso de esta, fue para resolver errores de lógica al momento de probar el código, para probar los algoritmos, detectar errores dentro de estos y resolverlos. Adicionalmente resultó muy útil para el cálculo de las complejidades, consideramos, que la Inteligencia artificial, es una herramienta útil para esto ya que ahorra el tiempo de realizar los cálculos. Finalmente, también fue útil a la hora de hacer consultas conceptuales sobre los temas tratados en el taller, resultando ser una herramienta muy útil para aclarar dudas de manera rápida y precisa.